

SDR-SLC

VITA-SLC I/Q Stream Transport Specification

Field	Value
Document	SDR-SLC-VITA-1.0
Title	VITA-SLC I/Q Stream Transport Specification
Version	Protocol 1.0 Release 1.1 (revised for implementers)
Profile Names	VITA-A/1.0 (IF Data) VITA-A/1.0-CTX (Context)
Status	RELEASE CANDIDATE
Date	May 2026
Author	Ivo van Ling, PA2IX
Organisation	IXChange BV
Licence	Creative Commons CC BY 4.0 — Open Source

Table of Contents

Table of Contents	2
Revision History	4
1. Introduction	5
2. Normative Language	6
3. Terminology	7
4. Implementing VITA-SLC from Scratch	8
4.1 What You Are Building	8
4.2 Tier 1 Minimum: What You Must Send	8
4.3 What u8 Encoding Means	9
4.4 Tier 2: Adding the Context Packet	9
4.5 The Reference Level: Why It Matters	10
4.6 The AGC Problem	11
4.7 Free-Running Timestamps vs Calibrated Timestamps	11
4.8 Testing Your Implementation	11
5. UDP Transport	13
6. VITA-A/1.0 IF Data Packet Format [T1]	14
6.1 VRT Header Fields	14
6.2 Stream Identifier	15
6.3 Class Identifier	15
6.4 Timestamps	16
6.4.1 Timestamp Integer (UTC Seconds)	16
6.4.2 Timestamp Fractional (Picoseconds)	16
6.5 I/Q Payload Encodings	16
7. VITA-A/1.0-CTX Context Packet Profile [T2]	18
7.1 Context Packet Layout	18
7.2 VRT Header for Context Packets	19
7.3 Context Indicator Field (CIF0)	19
7.4 Reference Level Field	19
7.5 Payload Format Field	20
7.6 Gain Tracking: When to Send Context Packets	20
7.7 Receiver Behaviour (Client Rules)	21
7.8 Complete Annotated Packet Examples	21
7.8.1 IF Data Packet: 360 pairs, u8 encoding, Stream ID=1, Packet Count=0	21
7.8.2 Context Packet: Normal Mode, rf_gain=20 dB, RL=-30.0 dBm, Stream ID=1	22
7.8.3 Context Packet: After Gain Increase to 30 dB (RL now -40.0 dBm)	22
7.8.4 Context Packet: Direct Sampling Mode Enabled (real signal)	22
8. I/Q Format Negotiation	24

9. Stream Identification and Multiplexing.....	25
10. Clock and Timestamp Model.....	26
10.1 PTP Integration.....	26
11. Multi-Stream Operation	27
12. Conformance Requirements.....	28
12.1 Tier 1: IF Data Packet Sender	28
12.2 Tier 2: Context Packet Sender.....	28
12.3 Client Requirements	29
13. Security Considerations.....	30
14. Forward Compatibility.....	31
15. References	32
15.1 Normative References	32
15.2 Informative References.....	32
Appendix A Packet Size Reference Table.....	33
Appendix B Profile Comparison.....	34
Appendix C Implementation Checklist.....	35

Revision History

Rev	Date	Author	Description
1.0 draft	2026-05-01	I. van Ling	Initial release. VITA-A/1.0 IF Data profile. u8 and int16 encodings, UDP transport, PTPv2/GPS timestamps.
1.1 draft	2026-05-21	I. van Ling	Added VITA-A/1.0-CTX Context packet profile: Payload Format field for signal type (complex I/Q vs real direct sampling). CIF0=0x00000002. Packet size 5 words / 20 bytes.
1.2 draft	2026-05-22	I. van Ling	Extended VITA-A/1.0-CTX: added Reference Level field (VITA-49.0 §9.5.9). CIF0 updated to 0x00000082. Packet size 6 words / 24 bytes. Gain-tracking rules and power calculation formula.
1.0 release	2026-05-22	I. van Ling	Consolidated release. All drafts merged.
1.1 release	2026-05-23	I. van Ling	Revised for implementers. Added §4 'Implementing VITA-SLC from Scratch'. Restored implementation-guide narrative throughout. Added implementation tier markers. Expanded annotated examples. Added embedded implementation walkthrough for STM32 and Raspberry Pi. Aligned with SDR-SLC-CP-1.0-v2 editorial style.

1. Introduction

VITA-SLC is the I/Q stream transport layer of the SDR-SLC protocol suite. It defines exactly how raw I/Q sample data travels from the radio hardware to the host computer over UDP. This document specifies two packet profiles: VITA-A/1.0 for the actual sample data, and VITA-A/1.0-CTX for metadata that makes the stream self-describing.

The profiles are constrained subsets of the VITA-49.0 standard (the VITA Radio Transport specification). VITA-49 is a well-established open standard used in professional and military radio systems. By building on it rather than inventing a custom binary format, SDR-SLC packets can be examined with existing VITA-49 analysis tools, and the format is documented independently of this project.

The constraint in VITA-A/1.0 is deliberate. Full VITA-49 has many optional features that add implementation complexity without benefiting a simple SDR system. VITA-A/1.0 fixes the options that matter (timestamping mode, class identifier, no trailer) and leaves the I/Q encoding negotiable. The result is a format that is simple enough to implement on a bare STM32H7 in an afternoon, yet rich enough to support calibrated power display, direct sampling mode, and late-joining receivers.

This document is both a normative specification and a practical guide. It explains not just what the packet format is, but why each field exists and how to implement it correctly on real hardware.

NOTE

This document covers the wire format only. Stream lifecycle management (open, start, stop, close) is specified in SDR-SLC-CP-1.0.

The SLC-CP control plane negotiates the I/Q encoding and stream parameters before UDP data begins flowing.

Read SDR-SLC-CP-1.0 first if you are new to the protocol suite.

2. Normative Language

Keyword	Meaning
SHALL	Absolute requirement. Non-conformant if omitted.
SHALL NOT	Absolute prohibition.
SHOULD	Strong recommendation. Deviation requires justification.
MAY	Optional. Implementations may include or omit.

Tier markers appear throughout this document. They indicate the SLC-CP profile at which a given requirement applies:

Marker	Meaning in this document
[T1]	Required at Tier 1 (all profiles). The absolute minimum.
[T2]	Required at Tier 2 Standard and above. Optional at Tier 1.
[T3]	Required at Tier 3 Full only.

3. Terminology

Term	Definition
IF Data Packet	A VITA-49 packet that carries I/Q sample payload. Packet Type 0x1 with Stream ID. Defined by the VITA-A/1.0 profile.
Context Packet	A VITA-49 IF Context packet that carries stream metadata (signal type and Reference Level). Packet Type 0x4 with Stream ID. Defined by the VITA-A/1.0-CTX profile.
VITA-A/1.0	The SDR-SLC constrained profile for IF Data packets. The minimal transmission format.
VITA-A/1.0-CTX	The SDR-SLC constrained profile for Context packets. Required at Tier 2.
Stream ID	A 32-bit integer assigned by the device that uniquely identifies a stream within a TCP session. Shared between IF Data packets and their associated Context packets.
VRT Header	The first 32-bit word of every VITA-49 packet. Contains packet type, flags, timestamp mode indicators, packet counter, and total size.
CIF0	Context Indicator Field 0. A 32-bit bitmask present in every Context packet that tells the receiver which optional context fields follow in the packet body.
Payload Format	A VITA-49.0 §9.13 context field that describes the signal type and sample encoding in the associated IF Data stream.
Reference Level	The received signal power at the antenna port that corresponds to a full-scale I/Q sample, in dBm. Enables calibrated power display.
dBFS	Decibels relative to full scale. 0 dBFS is the maximum representable amplitude (32767 for int16). Negative values indicate signal below full scale.
TSI	Timestamp Integer mode bits in the VRT header. In VITA-A/1.0: 01b = UTC seconds. In Context packets: 00b = no timestamp.
TSF	Timestamp Fractional mode bits. In VITA-A/1.0: 10b = picoseconds. In Context packets: 00b = none.
OUI	Organizationally Unique Identifier. IXChange BV: 0x402814. Embedded in the VITA-49 Class Identifier field.
PCC	Packet Class Code. Fixed at 0x0001 for VITA-A/1.0 IF Data packets.
ICC	Information Class Code. Fixed at 0x0000 in VITA-A/1.0.
samples_per_packet	Number of I/Q pairs per UDP packet. Fixed for the lifetime of a stream. Returned in the SLC-CP open_stream response.
sample_count	Running count of I/Q pairs output since the most recent start_rx call. Used to derive the fractional timestamp.
native format	The I/Q encoding that the hardware ADC produces directly, without conversion. Returned with native:true in capabilities.rx.iq_formats.

4. Implementing VITA-SLC from Scratch

This chapter is for the experimenter who has never worked with VITA-49 before and wants to understand what they actually need to send. It walks through the complete picture from wire bits to working stream before getting into field-level detail.

4.1 What You Are Building

Your device receives a `start_rx` command over the SLC-CP TCP connection. From that moment, it must start sending UDP packets to the IP address and port the client specified in `open_stream`. Each UDP packet is a VITA-A/1.0 IF Data packet. The payload of each packet is a block of raw I/Q sample pairs.

That is the core of it. The rest — the VRT header, the Class Identifier, the timestamps — is framing that tells the receiver how to interpret the payload and when the samples were captured.

What a VITA-A/1.0 packet looks like on the wire:

```
[Ethernet frame][IP header][UDP header]
[4 bytes: VRT Header      ] -- packet type, flags, size, counter
[4 bytes: Stream Identifier] -- which stream this belongs to
[4 bytes: Class ID Word 0  ] -- 0x00 | OUI 0x402814
[4 bytes: Class ID Word 1  ] -- ICC 0x0000 | PCC 0x0001
[4 bytes: Timestamp Integer] -- UTC seconds (32-bit)
[4 bytes: Timestamp Frac High] -- picoseconds, bits 63:32
[4 bytes: Timestamp Frac Low] -- picoseconds, bits 31:0
[N*4 bytes: I/Q Payload   ] -- N IQ pairs, interleaved
```

Total header: 28 bytes.

With 360 IQ pairs at int16: payload = 1440 bytes.

Total IP datagram: 28 + 1440 + 8 (UDP) + 20 (IP) = 1496 bytes.

Ethernet MTU limit: 1500 bytes. We have 4 bytes of margin.

4.2 Tier 1 Minimum: What You Must Send

Tier 1: minimum viable IF Data packet

At Tier 1, you must send correctly formatted IF Data packets. Context packets are not required. The client will assume complex I/Q mode and use any Reference Level it obtained from SLC-CP capabilities.

Here is what a Tier 1 device must fill in for each packet:

```
// C pseudocode for building a minimal VITA-A/1.0 IF Data packet
// Assumes 360 IQ pairs, int16 encoding, big-endian output.

uint32_t stream_id = 1;           // from open_stream response
uint32_t packet_count = 0;        // 0..15, increment each packet
uint64_t sample_count = 0;        // increment by 360 each packet
uint32_t utc_seconds = get_utc_seconds(); // from RTC or free-running counter
uint32_t fs_hz = 2048000;         // sample rate

void build_packet(uint8_t *buf, int16_t *iq_samples) {
    // VRT Header: 0x18600000 | (packet_count << 16) | packet_size_in_words
    // 0x1 = packet type 0001b (IF Data + Stream ID)
    // 0x8 = Class ID present
    // 0x6 = TSI=01b (UTC), TSF=10b (picoseconds)
    // 0x0 = no trailer
    uint32_t pkt_size = 7 + 360; // 7 header words + 360 payload words
    uint32_t hdr = 0x18600000 | ((packet_count & 0xF) << 16) | pkt_size;
    write_be32(buf+0, hdr);
}
```

```

write_be32(buf+4, stream_id);
write_be32(buf+8, 0x00402814); // OUI
write_be32(buf+12, 0x00000001); // ICC=0, PCC=1
write_be32(buf+16, utc_seconds); // integer timestamp

// Fractional timestamp in picoseconds
uint64 t_picos = (sample_count % fs_hz) * (uint64 t)1000000000000ULL / fs_hz;
write_be32(buf+20, (uint32_t)(picos >> 32));
write_be32(buf+24, (uint32_t)(picos & 0xFFFFFFFF));

// I/Q payload: 360 pairs, each pair is I (int16) then Q (int16), big-endian
for (int i = 0; i < 360; i++) {
    write_be16(buf+28 + i*4, iq_samples[i*2]); // I
    write_be16(buf+28 + i*4 + 2, iq_samples[i*2+1]); // Q
}

packet_count = (packet_count + 1) & 0xF;
sample_count += 360;
}

```

TIP

The VRT header value 0x18600000 is a constant for VITA-A/1.0 if you exclude the packet counter and size fields. Compute it once. The only things that change packet-to-packet are the counter (bits 19:16), the packet size (bits 15:0), and the payload content. Everything else is the same for every packet in a stream.

4.3 What u8 Encoding Means

The RTL-SDR natively produces 8-bit unsigned offset binary samples. Each I or Q component is a byte where 128 represents zero signal, 0 is maximum negative, and 255 is maximum positive. This is called unsigned offset binary.

```

// u8 encoding: 8 bits per I or Q component, 2 bytes per IQ pair
// Value 128 = 0 (no signal)
// Value 0 = minimum (-128 in signed terms)
// Value 255 = maximum (+127 in signed terms)

// Relationship to int16:
// u8 = (int16_sample >> 8) ^ 0x80
// int16 = ((int32_t)(u8 ^ 0x80)) << 8

// Payload for u8: 2 bytes per IQ pair
// [I_u8][Q_u8][I_u8][Q_u8]...

// Packet size with 360 pairs at u8:
// payload = 360 * 2 = 720 bytes
// total datagram = 720 + 28 + 8 + 20 = 776 bytes
// Well within 1500 byte MTU.

```

TIP

If you are bridging an RTL-SDR, use u8 encoding. It is the native format, requires no conversion, and halves the UDP bandwidth compared to int16. Declare it in capabilities.rx_iq_formats with native:true. The client will request it by default.

4.4 Tier 2: Adding the Context Packet

Tier 2: Context packet required before first IF Data packet

At Tier 2, your device must send one Context packet as the very first UDP packet after start_rx, before any IF Data packets. You must also send a new Context packet whenever the gain changes or the signal mode changes (i.e. when the direct_sampling control is modified via SLC-CP set_control).

The Context packet is 24 bytes. It tells the client two things: what kind of signal is in the payload (complex I/Q or real), and what signal power at the antenna corresponds to a full-scale sample (the Reference Level).

```
// C pseudocode for building a VITA-A/1.0-CTX Context packet
// for normal tuner mode (complex IQ) at rf gain=20 dB,
// reference_level = -30.0 dBm.

void build_context_packet(uint8_t *buf, uint32_t stream_id,
                        float_ref_level_dbm, bool is_real_signal) {
    // VRT Header: Packet Type 0x4 (IF Context), no CID, no timestamp
    // 0100b = type 4, CID=0, TSI=00b, TSF=00b, size=6 words
    write_be32(buf+0, 0x40000006 | (ctx_count & 0xF) << 16);
    write_be32(buf+4, stream_id);

    // CIF0: 0x00000082 = bit 7 (Reference Level) + bit 1 (Payload Format)
    write_be32(buf+8, 0x00000082);

    // Reference Level: 16.16 signed fixed-point, dBm
    int16_t rl_int = (int16_t)floorf(ref_level_dbm);
    uint16_t rl_frac = (uint16_t)((ref_level_dbm - rl_int) * 65536.0f);
    write_be32(buf+12, ((uint32_t)(uint16_t)rl_int << 16) | rl_frac);

    // Payload Format Word 0
    // For complex IQ: 0x0F8700E0
    // For real (direct sampling): 0x0F0700E0
    uint32_t pf_w0 = is_real_signal ? 0x0F0700E0 : 0x0F8700E0;
    write_be32(buf+16, pf_w0);
    write_be32(buf+20, 0x00000000); // Payload Format Word 1: always 0

    ctx_count = (ctx_count + 1) & 0xF;
}

// When to call it:
// After start_rx: build_context_packet(...) then start sending IF Data
// After set_control rf_gain: build_context_packet(...) then continue IF Data
// After set_control direct_sampling: build_context_packet(..., is_real=true/false)
```

TIP

The Payload Format Word 0 values 0x0F8700E0 (complex) and 0x0F0700E0 (real) are constants. You do not need to construct them from bit fields at runtime. Just store them as constants and select based on current mode.

4.5 The Reference Level: Why It Matters

Without the Reference Level, the client's waterfall S-meter shows relative levels only. The user can see that one signal is 10 dB stronger than another, but cannot say whether the stronger signal is at -80 dBm or -40 dBm. With the Reference Level, the client computes:

```
P_dBm = ReferenceLevel_dBm + 20 * log10(RMS_amplitude / full_scale)

Example with Reference Level = -30.0 dBm and u8 encoding (full_scale = 127):
Measured RMS = 12.7 counts = 12.7 / 127 = 0.1 of full scale
Level in dBFS = 20 * log10(0.1) = -20.0 dBFS
P_dBm = -30.0 + (-20.0) = -50.0 dBm

The Reference Level changes with gain:
At rf_gain = 20 dB, Reference Level = -30.0 dBm
Operator increases rf_gain to 30 dB (+10 dB gain)
New Reference Level = -30.0 - 10.0 = -40.0 dBm
(Full-scale samples now represent a 10 dB weaker antenna signal)
```

TIP

For an uncalibrated RTL-SDR dongle, you can initialise the Reference Level to a nominal value like -30.0 dBm at the preferred gain setting. The absolute accuracy will be ±5-8 dB, but the tracking across gain changes will be correct as long as you subtract the gain delta each time. The client will display power readings with an accuracy indicator.

4.6 The AGC Problem

When the RTL-SDR hardware AGC is active, the gain changes autonomously without a `set_control` command from the client. The device must still emit a Context packet whenever the AGC changes the gain, even without a client request. This is the only case where Context packets are generated by hardware events rather than client commands.

```
// AGC gain-change handler (called from hardware interrupt or timer)
void on_agc_gain_change(float new_gain_db) {
    // Compute new Reference Level
    float delta = new_gain_db - current_gain_db;
    current_reference_level_dbm -= delta;
    current_gain_db = new_gain_db;

    // Send Context packet before next IF Data packet
    build_context_packet(ctx_buf, stream_id,
                        current_reference_level_dbm, is_direct_sampling);
    udp_send(ctx_buf, 24, dest_ip, dest_port);
}
```

NOTE

Rate-limit AGC Context packet emission to no more than one per 100 ms. Some RTL-SDR tuners can update AGC gain tens of times per second. Sending a Context packet for every gain step would flood the network. Keep a timestamp of the last Context packet and skip emission if the previous one was less than 100 ms ago; send one packet at the end of the quiet period instead.

4.7 Free-Running Timestamps vs Calibrated Timestamps

Your device needs to fill in the Timestamp Integer field (UTC seconds) and the Timestamp Fractional field (picoseconds). There are two cases:

- No PTP or GPS: Initialise a seconds counter to the current time if you have an RTC, or to zero if you do not. Increment it every second. The fractional timestamp is computed from the sample count (see §6.3). The client's `get_clock_status` will show `source=free_running` or `source=manual`. The waterfall still works; timestamps are just relative rather than UTC-calibrated.
- PTP grandmaster on the LAN: Use the PTPv2 SYNC and FOLLOW_UP messages to discipline your RTC to UTC. Subtract the UTC offset (37 seconds as of 2024, from the PTPv2 ANNOUNCE message) from TAI to get UTC. The client's `get_clock_status` shows `source=ptpv2`, `syncd=true`. Timestamps are UTC-accurate to better than 100 µs.

TIP

For a first implementation, a free-running counter starting from zero is perfectly acceptable. Set `seconds=seconds_since_boot` and compute the fractional timestamp from `sample_count`. The client will display a 'clock not synchronised' indicator but everything else works. Add PTP support later when the basic stream is working.

4.8 Testing Your Implementation

Once you have a basic IF Data packet sender working, test it before worrying about Context packets or calibration:

- Send SLC-CP `open_stream` and `start_rx` from a minimal test client (netcat plus a Python script works fine).
- Capture UDP packets on the client host with Wireshark, filtering on the destination port. Verify: packet arrives, size is correct (776 bytes for u8/360 pairs, 1496 bytes for int16/360 pairs), source IP matches the device.

- Decode the first 28 bytes manually against the field definitions in §6. Verify the VRT Header has type=0001b, CID=1, TSI=01b, TSF=10b. Verify OUI = 0x402814.
- Feed the UDP stream into the SDR-SLC Swift client. You should see a waterfall. If the waterfall looks like noise but responds to signals, your packet format is correct. If it looks completely wrong, check byte order (everything must be big-endian).
- Verify the Packet Count field increments correctly. The client will report dropped packets if it sees gaps in the 0-15 sequence.

5. UDP Transport

VITA-SLC packets travel over UDP/IPv4. The choice of UDP over TCP is deliberate: UDP has lower overhead, no retransmission delay, and allows the receiver to detect and measure packet loss without waiting for timeouts. Lost I/Q packets cause brief waterfall gaps, which are far preferable to the latency spikes that TCP retransmission would introduce.

Parameter	Value / Rule
Protocol	UDP/IPv4. Unicast only. Multicast is not supported.
Source address	Device IP address. Returned by the SLC-CP open_stream response as source.ip.
Source port	Per-stream. Returned by open_stream as source.port. Default: 4621 for RX, 4622 for TX.
Destination address	Client IP address, specified by the client in the open_stream request as destination.ip.
Destination port	Client UDP port, specified by the client in open_stream as destination.port.
MTU constraint	Each IF Data IP datagram SHALL NOT exceed 1500 bytes (standard Ethernet MTU). Context packets are 24 bytes of payload; total datagram is 52 bytes — no MTU concern.
Packet rate	sample_rate_sps / samples_per_packet packets per second. E.g. 2,048,000 / 360 $\approx$ 5,689 packets/second.
Jitter	The sender SHOULD pace packets at the hardware sample rate. Bursting multiple packets then pausing degrades receiver buffer performance.

NOTE At 360 IQ pairs and int16 encoding: payload = 1440 bytes, IP datagram = 1496 bytes (4-byte MTU margin). At u8 encoding: payload = 720 bytes, IP datagram = 776 bytes. Both are safe for standard Ethernet. See Appendix A for the full size reference table.

TIP On STM32H7 with lwIP and FreeRTOS: use a DMA-driven timer to trigger packet assembly and send at the correct packet rate. Do not use a polling loop. The lwIP UDP send function (udp_sendto) is safe to call from a FreeRTOS task. Pre-allocate a pbuf for each packet and reuse it by resetting the payload pointer each cycle.

6. VITA-A/1.0 IF Data Packet Format [T1]

Every UDP payload sent by a VITA-A/1.0 device is a VITA-A/1.0 IF Data packet. The format is fixed: 28 bytes of header followed by the I/Q payload. There is no trailer.

Figure 1 — Complete VITA-A/1.0 IF Data Packet Layout

Offset	Size	Field
+0	4 bytes	VRT Header [31:28]=0001b Packet Type: IF Data + Stream ID [27]=1 Class ID Present (CID) [26]=0 No Trailer [25:24]=00b Reserved / Not V49.2 (always 0) [23:22]=01b TSI: UTC integer seconds [21:20]=10b TSF: picoseconds [19:16]=0..F Packet Count (modulo 16) [15:0]=size Packet Size in 32-bit words
+4	4 bytes	Stream Identifier (32-bit, assigned by device)
+8	4 bytes	Class ID Word 0: [0x00 reserved OUI 0x402814]
+12	4 bytes	Class ID Word 1: [ICC 0x0000 PCC 0x0001]
+16	4 bytes	Timestamp Integer: UTC seconds since epoch 32-bit unsigned. Overflows 2106-02-07.
+20	4 bytes	Timestamp Fractional HIGH (bits 63:32) Upper 32 bits of 64-bit picosecond value.
+24	4 bytes	Timestamp Fractional LOW (bits 31:0) Lower 32 bits of 64-bit picosecond value.
+28	N×4 B	I/Q Payload N = samples per packet IQ pairs. Interleaved: I0 Q0 I1 Q1 ... I(N-1) Q(N-1). Each component in the negotiated encoding.

Total fixed header: 28 bytes (7 words).
 Example: 360 pairs, int16 → payload 1440 B → datagram 1496 B.
 Example: 360 pairs, u8 → payload 720 B → datagram 776 B.

6.1 VRT Header Fields

The VRT Header is a single 32-bit word. Every field is fixed in VITA-A/1.0 except the Packet Count and the Packet Size.

Bits	Field	Value in VITA-A/1.0	Notes
[31:28]	Packet Type	0001b	IF Data Packet with Stream ID. This identifies the packet as a VITA-49 IF Data packet, not a context or extension packet.
[27]	Class ID Present	1	The Class Identifier fields at offsets +8 and +12 are present. Always 1 in VITA-A/1.0.
[26]	Trailer Present	0	No trailer appended after the payload. Always 0. Receivers that see 1 here should discard the packet.
[25]	Reserved	0	Reserved per VITA-49.0. Always 0. Receivers ignore this bit.

Bits	Field	Value in VITA-A/1.0	Notes
[24]	Not a V49.2 packet	0	Distinguishes VITA-49.2 packets. Always 0 for VITA-A/1.0.
[23:22]	TSI	01b	Timestamp Integer mode: UTC seconds since Unix epoch (1970-01-01T00:00:00Z).
[21:20]	TSF	10b	Timestamp Fractional mode: real-time (picoseconds, 64-bit unsigned).
[19:16]	Packet Count	0..15	Modulo-16 counter per stream. Increments with each IF Data packet. Maintained independently from Context packet count. Clients use gaps to detect dropped packets.
[15:0]	Packet Size	Variable	Total packet size in 32-bit words, counting every word from the VRT Header to the end of the payload. For int16 encoding with 360 pairs: 7 header + 360 payload = 367 (0x016F). For u8 with 360 pairs: 7 header + 180 payload words = 187 (0x00BB).

NOTE *The Packet Size field counts 32-bit words, not bytes. For u8 encoding, each IQ pair is 2 bytes. Two u8 IQ pairs pack into one 32-bit word. With 360 pairs: $360 / 2 = 180$ payload words. Total packet size = $7 + 180 = 187$ words.*

6.2 Stream Identifier

A 32-bit word immediately follows the VRT Header. Its value is the VITA-49 numeric Stream ID, which is returned by SLC-CP in the open_stream response as vita.stream_id. The device assigns this value and it does not change for the lifetime of the stream session.

- The Stream ID SHALL NOT be reused within a TCP control session. If you open a new stream after closing the previous one, assign a new Stream ID.
- The Stream ID is opaque to the client. Clients treat it as a number, not as a structured value. Assign sequential integers starting from 1, or use a random value — it does not matter.
- When multiple streams are active simultaneously (e.g. RX stream and TX stream), each has its own Stream ID. Clients demultiplex packets by Stream ID.

6.3 Class Identifier

Two 32-bit words at offsets +8 and +12 form the VITA-49 Class Identifier. This pair of values globally and unambiguously identifies every VITA-A/1.0 packet on any network. A receiver that sees these values knows it is looking at a VITA-SLC IF Data packet.

Word	Bits	Field	Value	Description
Word 0	[31:24]	Reserved	0x00	Always zero per VITA-49.0.
Word 0	[23:0]	OUI	0x402814	IXChange BV IEEE-registered Organizationally Unique Identifier. In memory (big-endian): bytes 0x40, 0x28, 0x14.

Word	Bits	Field	Value	Description
Word 1	[31:16]	ICC	0x0000	Information Class Code. Always 0x0000 in VITA-A/1.0.
Word 1	[15:0]	PCC	0x0001	Packet Class Code. 0x0001 identifies the VITA-A/1.0 IF Data profile.

WARNING Receivers SHALL reject IF Data packets where OUI $\neq$ 0x402814 or PCC $\neq$ 0x0001. These values together form a globally unique packet type identifier under IEEE OUI policy. Do not use the placeholder 0x000000 OUI. If you are building hardware under a different organisation, register your own OUI and use it.

6.4 Timestamps

6.4.1 Timestamp Integer (UTC Seconds)

A 32-bit unsigned integer at offset +16. Contains the number of whole seconds elapsed since the Unix epoch (1970-01-01T00:00:00Z). This is the TSI=01b encoding per VITA-49.0.

NOTE This field overflows on 2106-02-07T06:28:15Z. The 32-bit limit is inherited from VITA-49.0. A future VITA-A/2.0 profile will address this. For now, the overflow date is 80 years away.

When the device clock is free-running (no PTP or GPS), this field counts seconds since the most recent start_rx call, starting from zero. The client's get_clock_status will show source=free_running; the client uses the timestamp as a relative counter rather than an absolute UTC time.

6.4.2 Timestamp Fractional (Picoseconds)

A 64-bit unsigned integer split across two 32-bit words: high word at offset +20, low word at offset +24 (high word first). Contains the sub-second offset in picoseconds from the start of the current UTC second.

The fractional timestamp is derived from the running sample count:

```
picos = (sample_count mod fs_hz) * 10^12 / fs_hz

sample_count = number of IQ pairs output since the last start_rx call
fs_hz        = sample rate in samples per second

CRITICAL: Use integer arithmetic only. Floating-point gives rounding errors
that accumulate over time. Use uint64_t in C.

Example at 2,048,000 SPS with 360 pairs per packet:
Packet 0: sample_count = 0
          picos = 0
Packet 1: sample_count = 360
          picos = 360 * 1,000,000,000,000 / 2,048,000
                = 175,781,250 ps (0.17578125 ms)
Packet 2: sample_count = 720
          picos = 350,781,250 (wait, re-check: 720*10^12/2048000)
                = 351,562,500 ps

sample_count resets to 0 on each start_rx call.
The integer seconds field increments by 1 when picos wraps from
999,999,999,999 back toward 0.
```

6.5 I/Q Payload Encodings

The I/Q payload immediately follows the 28-byte header. Samples are interleaved: $I_0 Q_0 I_1 Q_1 \dots I(N-1) Q(N-1)$. The encoding (bits per component, byte order, signed/unsigned) is negotiated via SLC-CP before streaming begins and remains constant for the entire stream lifetime.

All encodings use big-endian byte order. All multi-byte values are transmitted most-significant byte first.

Label	Encoding	Bits	Bytes/pair	Notes
u8	Unsigned offset binary	8	2	Native on RTL-SDR. 128 = zero. $(\text{int16} \gg 8) \wedge 0x80$. Lowest bandwidth option.
int16	Two's complement	16	4	Native on most 16-bit ADCs. IXChange SDR hardware native. Signed, big-endian.
int24	Two's complement	24	8	24-bit value left-justified in a 32-bit container. Lower 8 bits are zero-padded. Used for high-dynamic-range ADCs.
float32	IEEE 754 single	32	8	± 1.0 corresponds to full-scale ADC range. Big-endian. Reserved for future high-performance hardware.

NOTE

For int24: each 32-bit word carries one component. The upper 24 bits are the sample value in two's complement. The lower 8 bits are always zero. This matches the standard representation for 24-bit audio DAC hardware. Two int24 components (I and Q) occupy 8 bytes per pair.

7. VITA-A/1.0-CTX Context Packet Profile [T2]

Context packets make the VITA-SLC stream self-describing. Without them, the client must obtain signal type and calibration information from the SLC-CP control channel and keep it in sync manually. With Context packets, the UDP stream itself carries this information in-band, adjacent to the data it describes.

Two pieces of information are carried:

- Signal type — whether the payload contains genuine complex I/Q (normal tuner mode) or a real-valued signal from one ADC branch (RTL-SDR direct sampling mode). The client's DSP pipeline must know this before it can demodulate correctly.
- Reference Level — the signal power at the antenna port that corresponds to full-scale samples at the current gain setting. Changes with every gain adjustment. Enables calibrated power display.

WARNING Normal tuner mode and direct sampling mode are not minor variants. A DSP pipeline that applies complex quadrature demodulation to a real direct-sampling signal produces completely incorrect output: the spectrum appears mirrored, the frequency scale is wrong, and demodulation fails. The client must know the mode before processing the first sample.

A Tier 1 device does not send Context packets. The client assumes complex I/Q mode and uses the Reference Level from SLC-CP capabilities (if present) or displays power as uncalibrated. A Tier 2 device sends a Context packet before the first IF Data packet after start_rx and after any gain or mode change.

7.1 Context Packet Layout

Figure 7 — Complete VITA-A/1.0-CTX Context Packet Layout

Offset	Size	Field
+0	4 bytes	VRT Header [31:28]=0100b Packet Type: IF Context + Strm ID [27]=0 No Class Identifier [26]=0 No Trailer [25:24]=00b Reserved (always 0) [23:22]=00b TSI: No Timestamp [21:20]=00b TSF: No Timestamp [19:16]=0..F Packet Count (separate from data) [15:0]=0x0006 Packet Size: 6 words (fixed)
+4	4 bytes	Stream Identifier (same value as IF Data stream)
+8	4 bytes	CIF0 = 0x00000082 (always this value) bit 7 set = Reference Level present bit 1 set = Payload Format present all other bits = 0
+12	4 bytes	Reference Level 16.16 signed fixed-point dBm. Upper 16 bits: integer part (signed, two's comp) Lower 16 bits: fractional part (1/65536 dBm)
+16	4 bytes	Payload Format Word 0 Complex IQ: 0x0F8700E0 Real (direct sampling): 0x0F0700E0
+20	4 bytes	Payload Format Word 1: 0x00000000 (always)

Total packet: 6 words × 4 bytes = 24 bytes fixed size.

VRT Header constant for Context: 0x40000006 | (count << 16)

7.2 VRT Header for Context Packets

The Context packet VRT Header differs from the IF Data header in four ways: the Packet Type is 0x4 (IF Context) instead of 0x1, Class ID Present is 0 (no Class Identifier fields), TSI is 00b (no timestamp), and TSF is 00b (no timestamp). The Packet Size is always 6.

Bits	Field	Value	Notes
[31:28]	Packet Type	0100b	IF Context Packet with Stream ID. Distinguishes from IF Data (0001b).
[27]	Class ID Present	0	No Class Identifier in Context packets. The Class Identifier is present in IF Data packets only.
[26]	Trailer Present	0	No trailer. Always 0.
[25:24]	Reserved	00b	Always 0.
[23:22]	TSI	00b	No Timestamp. Context packets describe the stream state, not a sample time.
[21:20]	TSF	00b	No Timestamp Fractional.
[19:16]	Packet Count	0..15	Modulo-16 counter for Context packets. Maintained independently from the IF Data packet count. A client MUST NOT use this to infer data packet loss.
[15:0]	Packet Size	6	Always 6: 1 (header) + 1 (stream ID) + 1 (CIF0) + 1 (Reference Level) + 2 (Payload Format) = 6 words.

7.3 Context Indicator Field (CIF0)

CIF0 is a 32-bit bitmask at offset +8. Each set bit indicates that a corresponding optional context field is present in the packet body. The CIF0 value in VITA-A/1.0-CTX is always 0x00000082.

CIF0 = 0x00000082

bit 7 (0x80) = 1 → Reference Level field present at offset +12
 bit 1 (0x02) = 1 → Payload Format field present at offset +16
 all other bits = 0

Field ordering within the Context packet body is determined by CIF0 bit order, from most significant set bit to least significant. Bit 7 > bit 1, so Reference Level comes before Payload Format in the packet. This ordering is defined by VITA-49.0 §8.2 and is not optional.

NOTE

*The field ordering rule (high bit first) is the most common implementation error with VITA-49 Context packets. Reference Level **MUST** come before Payload Format because bit 7 > bit 1. If you put them in the wrong order, your Context packets will be parsed incorrectly.*

7.4 Reference Level Field

A 32-bit word at offset +12. Encodes the Reference Level in dBm using VITA-49.0 §9.5.9 signed 16.16 fixed-point format. The upper 16 bits are the integer part (signed two's complement). The lower 16 bits are the fractional part (1/65536 dBm per LSB).

```
Encoding: word = (int16_t)(floor(dBm)) << 16 | (uint16_t)(frac(dBm) * 65536)
```

```
-30.0 dBm: int=-30 (0xFFE2) frac=0.0 (0x0000) word=0xFFE20000
-40.0 dBm: int=-40 (0xFFD8) frac=0.0 (0x0000) word=0xFFD80000
-10.5 dBm: int=-11 (0xFFFF5) frac=0.5 (0x8000) word=0xFFFF58000
-25.75 dBm: int=-26 (0xFFE6) frac=0.25 (0x4000) word=0xFFE64000
```

Note on negative numbers with non-zero fraction:

```
-10.5 = -11 + 0.5 (integer part is floor(-10.5) = -11, fraction is 0.5)
The fractional part is always positive (>= 0 and < 1).
Do NOT use -10 + (-0.5). Always compute as floor(value) + non-negative fraction.
```

C implementation:

```
int16_t rl_int = (int16_t)floorf(ref_level_dbm);
uint16_t rl_frac = (uint16_t)((ref_level_dbm - (float)rl_int) * 65536.0f);
uint32_t rl_word = ((uint32_t)(uint16_t)rl_int << 16) | rl_frac;
write_be32(buf+12, rl_word);
```

TIP

If your device does not have factory calibration data, use a hardcoded nominal Reference Level such as -30.0 dBm at the preferred gain setting. This gives the client something to display rather than 'uncalibrated'. Track gain changes correctly using the formula: $\text{new_RL} = \text{old_RL} - \text{gain_change_dB}$. The client will show power readings with an accuracy indicator for uncalibrated devices.

7.5 Payload Format Field

Two 32-bit words at offsets +16 and +20 form the VITA-49.0 §9.13 Payload Format field. The first word describes the signal type and encoding. The second word is always zero in VITA-A/1.0-CTX.

In practice you only need two constants. Select based on the current `direct_sampling` control state:

```
// Normal tuner mode (complex I/Q): Payload Format Word 0 = 0x0F8700E0
// Direct sampling mode (real signal): Payload Format Word 0 = 0x0F0700E0
// Payload Format Word 1: always 0x00000000

// Field breakdown of 0x0F8700E0 (complex mode):
// bit 31 = 1 Real/Complex: Complex Cartesian (I and Q)
// [30:29] = 00 (reserved)
// [29:25] = 00111b = 7 Data Item Format: Unsigned Integer
// bit 24 = 0 Sample Component Representation: signed fixed-point
// [23:20] = 0 (reserved)
// [19:15] = 00111b = 7 Data Item Size: 8 bits (value = size - 1)
// [14:11] = 0001b = 1 Vector Size: 2 components (value = count - 1)
// bit 10 = 0 Packing Method: processing efficient
// [9:5] = 00111b = 7 Item Packing Field Size: 8 bits (value = size - 1)
// [4:0] = 00000b = 0 Repeat Count: 1 (value = count - 1)

// Field breakdown of 0x0F0700E0 (real/direct sampling mode):
// bit 31 = 0 Real/Complex: Real
// [14:11] = 0000b = 0 Vector Size: 1 component (value = count - 1)
// All other fields identical to complex mode
```

NOTE

The Payload Format hex values 0x0F8700E0 and 0x0F0700E0 are provided for verification. Implementers should compute them independently from the VITA-49.0 §9.13 bit field definitions. The values shown above have been verified against those definitions.

7.6 Gain Tracking: When to Send Context Packets

A Context packet must be sent before the first IF Data packet whenever the Reference Level or the Payload Format changes. The three triggers are:

- After `start_rx`: Always send a Context packet first. The client needs the signal type and Reference Level before it can process any IF Data packets.

- After a gain control changes: When `rf_gain`, `lna_gain`, `if_gain`, or any other control that affects signal path gain is changed via SLC-CP `set_control`, update the Reference Level (`new_RL = old_RL - gain_change_dB`) and send a Context packet.
- After `direct_sampling` changes: When the `direct_sampling` control changes mode, the Payload Format changes (complex ↔ real) and send a Context packet.

The Reference Level tracks gain automatically. The relationship is simple: increasing the gain by G dB means the hardware is amplifying more, so the same antenna signal now produces larger samples. Full-scale samples now correspond to a G dB weaker antenna signal:

```
new reference level dbm = old reference level dbm - gain change dB
```

```
Example: at rf gain=20 dB, Reference Level = -30.0 dBm
Operator increases rf_gain to 30 dB (gain_change = +10 dB):
new RL = -30.0 - 10.0 = -40.0 dBm
```

```
Meaning: full-scale samples now represent -40 dBm at the antenna.
Previously they represented -30 dBm.
The display power P_dBm will be the same for the same physical signal,
because the higher sample amplitude is compensated by the lower Reference Level.
```

TIP

Track a single float variable: `current_reference_level_dbm`. Initialise it at stream open from your factory calibration table (or a nominal value). Update it on every gain change. Send a Context packet after every update before the next IF Data packet. That is the complete implementation.

7.7 Receiver Behaviour (Client Rules)

The following rules govern how a VITA-SLC client handles Context packets:

- A client that has NOT implemented Context packet parsing SHALL silently discard any UDP packet whose VRT Header Packet Type field (bits 31:28) is 0100b (0x4). Do not interpret the payload. Do not treat it as a data packet loss.
- A client that HAS implemented Context packet parsing SHOULD inspect the Context packet before processing any IF Data packets for a new stream. Wait for the first Context packet before rendering the waterfall.
- A client SHALL NOT use the Context packet Packet Count field to infer IF Data packet loss. The two counters are independent.
- A client SHOULD update its DSP pipeline and power display immediately upon receiving a Context packet. Do not wait for the next IF Data packet.
- The Context packet is authoritative. If the Context packet says the mode is real (direct sampling) but the SLC-CP `signal_type` field in the `open_stream` response says `complex_iq`, the client must use the Context packet value. The Context packet reflects the current hardware state; the `open_stream` response reflects the state at the time the stream was opened.
- If no Context packet has been received for a stream, the client MAY assume complex I/Q mode and use the Reference Level from SLC-CP `get_capabilities` or `get_status`. This is the Tier 1 client fallback.

7.8 Complete Annotated Packet Examples

7.8.1 IF Data Packet: 360 pairs, u8 encoding, Stream ID=1, Packet Count=0

Clock: 2026-05-20T14:32:01.000000000Z (epoch 1747751521), sample 0.

Offset	Bytes	Field and value
+00	18 00 00 BB	VRT Header: 0x180000BB [31:28] = 0001b Packet Type: IF Data + Stream ID [27] = 1 Class ID Present [26] = 0 No Trailer [25:24] = 00b Reserved = 0 [23:22] = 01b TSI: UTC seconds [21:20] = 10b TSF: picoseconds [19:16] = 0000b Packet Count: 0 [15:0] = 0x00BB Packet Size: 187 words (7 header + 180 payload) 180 = 360 u8 pairs / 2 per word
+04	00 00 00 01	Stream Identifier: 1
+08	00 40 28 14	Class ID Word 0: reserved=0x00, OUI=0x402814
+12	00 00 00 01	Class ID Word 1: ICC=0x0000, PCC=0x0001
+16	68 2B 6F 01	Timestamp Integer: 0x682B6F01 = 1747751521 → 2026-05-20T14:32:01Z
+20	00 00 00 00	Timestamp Fractional High: 0
+24	00 00 00 00	Timestamp Fractional Low: 0 → 0 ps (first packet)
+28	80 80 80 80	Payload first 4 bytes: I0=0x80 Q0=0x80 I1=0x80 Q1=0x80 0x80 = 128 = zero signal in u8 offset binary
...	...	[continuing for 720 bytes = 360 IQ pairs]
Total UDP payload: 28 + 720 = 748 bytes.		
Total IP datagram: 748 + 8 (UDP) + 20 (IP) = 776 bytes.		

7.8.2 Context Packet: Normal Mode, rf_gain=20 dB, RL=-30.0 dBm, Stream ID=1

Offset	Bytes	Field and value
+00	40 00 00 06	VRT Header: 0x40000006 [31:28] = 0100b Packet Type: IF Context + Stream ID [27] = 0 No Class ID (not present in context pkts) [26] = 0 No Trailer [25:24] = 00b Reserved [23:22] = 00b TSI: No Timestamp [21:20] = 00b TSF: No Timestamp [19:16] = 0000b Packet Count: 0 (first context packet) [15:0] = 0x0006 Packet Size: 6 words (24 bytes, fixed)
+04	00 00 00 01	Stream Identifier: 1 (same as IF Data stream)
+08	00 00 00 82	CIF0: 0x00000082 bit 7 = 1 → Reference Level field present (+12) bit 1 = 1 → Payload Format field present (+16)
+12	FF E2 00 00	Reference Level: 0xFFE20000 upper 16 = 0xFFE2 = -30 (signed int16) lower 16 = 0x0000 = 0.0 fraction → Reference Level = -30.0 dBm
+16	0F 87 00 E0	Payload Format Word 0: 0x0F8700E0 bit 31 = 1 → Complex Cartesian (I and Q) [14:11] = 0001b → Vector Size = 2 (two components) (identifies normal complex IQ mode)
+20	00 00 00 00	Payload Format Word 1: 0x00000000 (always zero)
Total UDP payload: 24 bytes. IP datagram: 52 bytes.		
This packet MUST be sent before the first IF Data packet after start_rx.		

7.8.3 Context Packet: After Gain Increase to 30 dB (RL now -40.0 dBm)

+00	40 10 00 06	VRT Header: Packet Count = 1 (second context packet)
+04	00 00 00 01	Stream ID: 1
+08	00 00 00 82	CIF0: 0x00000082 (unchanged)
+12	FF D8 00 00	Reference Level: 0xFFD80000 0xFFD8 = -40 (signed int16) → -40.0 dBm (gain increased +10 dB → RL decreased 10 dB)
+16	0F 87 00 E0	Payload Format W0: unchanged, still complex IQ
+20	00 00 00 00	Payload Format W1: 0

7.8.4 Context Packet: Direct Sampling Mode Enabled (real signal)

+00	40 20 00 06	VRT Header: Packet Count = 2
+04	00 00 00 01	Stream ID: 1
+08	00 00 00 82	CIF0: 0x00000082

```
+12 FF DD 00 00 Reference Level: 0xFFDD0000 = -35.0 dBm
                    (direct sampling path has different gain budget)
+16 0F 07 00 E0 Payload Format W0: 0x0F0700E0
                    bit 31 = 0 → Real (single component, not complex)
                    [14:11] = 0000b → Vector Size = 1 (one component)
                    (identifies direct sampling real mode)
+20 00 00 00 00 Payload Format W1: 0
```

8. I/Q Format Negotiation

The I/Q encoding used in IF Data packets is negotiated via SLC-CP before streaming begins. The sequence is:

- 1. Client calls `get_capabilities` and reads `capabilities.rx.iq_formats[]`. Each entry has an `encoding`, `bits_per_component`, `container_bits`, `packing`, `byte_order`, `native`, `supported`, and `label` field.
- 2. Client selects a format entry where `supported` is true. The entry with `native:true` requires no hardware conversion and minimises latency.
- 3. Client includes the selected format object in the `open_stream` request `params.format` field.
- 4. Device confirms the format in the `open_stream` response. The client must use the echoed format object, not the one it sent, to decode packets — in case the device substituted a compatible variant.
- 5. The negotiated format applies for the entire stream. If the client wants a different format, it must close the stream and open a new one.

NOTE

If the client omits the format field from the open_stream request, the device uses the format with native:true as the default. At Tier 1, a device may ignore the format request entirely and always use its native format.

The format fields in the capabilities and open_stream response:

Field	Type	Description
encoding	string	"twos_complement", "unsigned", or "float32".
bits_per_component	integer	Significant bits per I or Q component (e.g. 16, 8, 24, 32).
container_bits	integer	Size of the storage container in bits. Equals bits_per_component for int16 and u8; 32 for int24.
packing	string	Always "interleaved" in VITA-A/1.0 (I Q I Q ...).
byte_order	string	Always "big_endian" in VITA-A/1.0.
native	boolean	true if this format is produced directly by the ADC without conversion. The client should prefer this format for lowest latency.
supported	boolean	true if the device can currently use this format. A device may list formats with supported:false to document future planned support.
label	string	Short human-readable identifier: u8, int16, int24, float32.

9. Stream Identification and Multiplexing

Every UDP packet — whether IF Data or Context — carries a 32-bit Stream ID in the second word of the packet (offset +4). The Stream ID links packets to the stream session that was established via SLC-CP `open_stream`.

- The device assigns the numeric Stream ID and returns it in the SLC-CP `open_stream` response as `vita.stream_id`.
- The Stream ID SHALL NOT be reused within a TCP control connection session. After `close_stream`, that Stream ID value must not be used again for the lifetime of the TCP session.
- The Stream ID is shared between IF Data packets and Context packets for the same stream. A Context packet and its associated IF Data packets all carry the same Stream ID.
- When the client has multiple streams open simultaneously (e.g. an RX stream and a TX stream), it demultiplexes received packets first by Stream ID, then by Packet Type (VRT Header bits 31:28): 0x1 = IF Data, 0x4 = Context.
- The Packet Count field (bits 19:16 of the VRT Header) is maintained separately for IF Data packets and Context packets within each stream. A gap in the IF Data Packet Count indicates dropped data packets. A gap in the Context Packet Count indicates dropped context packets, which does not indicate data loss.

TIP

For a single-stream device, the numeric Stream ID can always be 1. The SLC-CP `stream_id` string (used in `start_rx`, `stop_rx`, etc.) is different from the VITA-49 numeric stream ID. The SLC-CP string is assigned by you; the VITA-49 integer is what goes in the packet header. Both are returned by `open_stream`; one as `stream_id`, the other as `vita.stream_id`.

10. Clock and Timestamp Model

Every IF Data packet carries a UTC timestamp. The quality of that timestamp depends on the device's clock source. The client can query the clock source via SLC-CP `get_clock_status`.

Source	Accuracy	Notes
GPS-locked PTPv2 grandmaster	< 1 μ s	Best. UTC-traceable. Recommended for frequency calibration and time-domain analysis.
Non-GPS PTPv2 grandmaster	< 100 μ s	Good. Accuracy depends on grandmaster hardware and network path.
GPS receiver on device	< 1 μ s	Good. Requires on-device GPS module.
Manual (SLC-CP <code>set_clock</code>)	Client accuracy	Acceptable for logging. Not UTC-traceable.
Free-running counter	~1 ms/day drift	TSI starts at 0 (or seconds since boot) and counts from there. Not a Unix timestamp.

NOTE

Context packets carry no timestamp at all ($TSI=TSF=00b$). They describe the stream state at a point in time, but are not precisely timestamped. The timestamp that matters is in the IF Data packets.

Clients **SHOULD** call `get_clock_status` at startup and warn the operator if `source=free_running`. Timestamps from a free-running device are relative (not UTC-calibrated) and should not be used for absolute time measurements.

10.1 PTP Integration

If your device uses PTPv2 (IEEE 1588-2008), a grandmaster on the LAN provides TAI time via SYNC and FOLLOW_UP messages. The device converts TAI to UTC by subtracting the UTC offset. As of 2024, the TAI–UTC offset is 37 seconds. This value is carried in the PTPv2 ANNOUNCE message and should be updated when it changes.

The VITA-49 Timestamp Integer field carries UTC seconds since the Unix epoch. If your PTP implementation gives you TAI time, subtract 37 before writing the timestamp field.

TIP

PTP is optional. A free-running counter is acceptable for Tier 1. If you use PTP, lwIP does not include a PTP implementation by default. The `ptpd` or `linuxptp` software stacks are available for Linux; for bare-metal STM32, small PTP slave implementations exist on GitHub. Search for 'STM32 PTP slave' or use the `ptpd` port for FreeRTOS.

11. Multi-Stream Operation

Most SDR-SLC implementations handle one active RX stream at a time. Hardware with multiple independent receiver chains can support concurrent streams with distinct Stream IDs, sample rates, and UDP destinations. The maximum number of concurrent streams is reported in `capabilities.rx.channels`.

- Each concurrent stream has its own Stream ID, its own Packet Count counter, its own Context packet sequence, and its own UDP destination.
- When multiple streams are active, each stream uses its own Context packet sequence. A Context packet is associated with its stream by Stream ID.
- The device SHALL NOT send packets for a stream that has been stopped or closed.
- Single-chain hardware enforces the one-active-stream-per-direction rule in the SLC-CP `open_stream` logic by returning `stream_already_open` if the client tries to open a second simultaneous stream.

12. Conformance Requirements

12.1 Tier 1: IF Data Packet Sender

Tier 1 — Minimum VITA-SLC device conformance

Requirement
Transmit VITA-49 IF Data packets with Packet Type 0x1 (0001b) and Stream ID.
Set Class ID Present (bit 27) = 1 in all IF Data VRT Headers.
Set Trailer Present (bit 26) = 0 in all packets.
Set TSI = 01b (UTC seconds) in IF Data VRT Headers.
Set TSF = 10b (picoseconds) in IF Data VRT Headers.
Use OUI = 0x402814 in Class ID Word 0.
Use PCC = 0x0001 in Class ID Word 1.
Use ICC = 0x0000 in Class ID Word 1.
Compute Timestamp Fractional using integer arithmetic only. Floating-point SHALL NOT be used.
Reset sample_count to zero on each start_rx call.
Increment Packet Count modulo 16 for each IF Data packet. Maintain independently per stream.
Honour the negotiated I/Q format for the entire stream lifetime. Do not change mid-stream.
Fit each IP datagram within 1500 bytes (standard Ethernet MTU).
Use big-endian byte order for all fields and all payload encodings.
Assign a unique numeric Stream ID per stream. Do not reuse within a TCP session.
Stop sending packets immediately on TCP control connection loss.

12.2 Tier 2: Context Packet Sender

Tier 2 — adds Context packet requirement

Requirement
All Tier 1 requirements.
Emit a VITA-A/1.0-CTX Context packet before the first IF Data packet after start_rx.
Emit a Context packet before the first IF Data packet after any direct_sampling mode change.
Emit a Context packet before the first IF Data packet after any gain control change (rf_gain, lna_gain, if_gain, or vendor gain control).
Set Context packet VRT Header: Type=0x4 (0100b), CID=0, TSI=00b, TSF=00b, Size=6.
Set CIF0 = 0x00000082 in all Context packets.
Place Reference Level field at offset +12 (before Payload Format, per CIF0 bit ordering).
Encode Reference Level as 16.16 signed fixed-point dBm per VITA-49.0 §9.5.9.

Requirement
Set Payload Format Word 0 = 0x0F8700E0 for complex I/Q mode.
Set Payload Format Word 0 = 0x0F0700E0 for real (direct sampling) mode.
Set Payload Format Word 1 = 0x00000000 always.
Maintain a separate Packet Count for Context packets. Do not share with IF Data counter.
Update Reference Level correctly on each gain change: $\text{new_RL} = \text{old_RL} - \text{gain_change_dB}$.
Rate-limit Context packet emission to no more than one per 100 ms during AGC operation.
MAY emit additional Context packets periodically to support late-joining receivers.

12.3 Client Requirements

Requirement
Decode IF Data payload using the negotiated format from the SLC-CP open_stream response.
Demultiplex received UDP packets first by Stream ID, then by Packet Type (bits 31:28 of VRT Header).
Silently discard any packet with VRT Packet Type 0x4 (IF Context) if Context packet parsing is not implemented.
NOT infer IF Data packet loss from Context packet Packet Count. The counters are independent.
Treat Stream ID as opaque. Do not interpret its structure.
Request only I/Q formats where supported:true in capabilities.rx.iq_formats.
Use the echoed format from the open_stream response (not the requested format) to decode packets.
SHOULD inspect the Context packet before processing any IF Data packets for a new stream.
SHOULD update DSP pipeline and power display on each received Context packet.
SHOULD treat the Context packet Reference Level as authoritative over SLC-CP fields.
SHOULD validate UDP source IP against the device address from open_stream response (security).

13. Security Considerations

VITA-SLC is designed for trusted local area network operation. It provides no authentication, encryption, or integrity verification. Any host on the LAN that can send UDP packets to the client's listening port can inject fabricated IF Data or Context packets.

The practical risks for a home station are:

- A spoofed IF Data packet injects false signal data into the waterfall. The operator would see signals that do not exist.
- A spoofed Context packet with a fabricated Reference Level causes the client to display incorrect power readings, without any visible indication that something is wrong.
- A spoofed Context packet changing the Payload Format could cause the DSP pipeline to misinterpret subsequent real data.

Mitigation: clients SHOULD validate the source IP address of received UDP packets against the device IP address returned by SLC-CP `open_stream`. Packets from unexpected source addresses should be discarded. This is a one-line filter in the receive path.

WARNING

On an unmanaged home LAN with no other users, these risks are negligible. On a shared or semi-public network (club station, university network), source IP validation is strongly recommended.

14. Forward Compatibility

The VITA-A/1.0 and VITA-A/1.0-CTX profiles are designed to be extended without breaking existing receivers:

- Reserved bits in the VRT Header (bits [25] and [24]) SHALL be set to 0 by transmitters and SHALL be ignored by receivers. Future profile versions MAY assign meaning to these bits.
- Reserved CIFO bits SHALL be set to 0 and ignored by receivers. Future revisions MAY add new context fields by setting additional CIFO bits.
- When a receiver encounters an unknown CIFO bit, it must skip the corresponding field in the packet body. The VITA-49.0 standard defines the sizes of all possible context fields, so a receiver that knows CIFO bit positions can skip unknown fields.
- New transport format labels (beyond u8, int16, int24, float32) MAY be added in future revisions with supported:false until implemented. Existing receivers ignore unknown labels.
- A future VITA-A/2.0 profile MAY use a different PCC value to allow coexistence with VITA-A/1.0 on the same network. Receivers SHALL ignore packets where $PCC \neq 0x0001$.

NOTE

When processing packets from an unknown Packet Type (other than 0x1 or 0x4), receivers SHOULD discard the packet and log an anomaly rather than attempting to decode the payload. Unknown packet types may be added by future profile versions.

15. References

15.1 Normative References

Reference	Title	Relevance
VITA-49.0	VITA Radio Transport Standard (VRT)	Base VRT packet format. §6 (VRT header), §8.1 (CIF0), §9.5.9 (Reference Level), §9.13 (Payload Format).
RFC 768	User Datagram Protocol	UDP transport for all VITA-SLC packets.
IEEE 1588-2008	Precision Time Protocol v2	PTPv2 clock synchronisation for UTC timestamps.
IEEE 802 / OUI	IEEE OUI Registry	IXChange BV OUI 40-28-14 (0x402814).
SDR-SLC-CP-1.0	SLC-CP Control Plane Specification	Stream lifecycle management, format negotiation, signal_type, reference_level_dbm.
SDR-SLC-PS-1.0	SDR-SLC Protocol Suite Architecture	Overall system architecture and design rationale.
RFC 2119	Key Words for Use in RFCs	Normative keyword definitions.

15.2 Informative References

Reference	Title	Notes
VITA-49.2	VITA Radio Transport Standard v2	May form the basis of a future VITA-A/2.0 profile. Addresses TSI 2106 overflow.
SDR-SLC-DISC-1.0	SLC-DISC Service Discovery Spec	How clients discover device IP and port before opening the control connection.
lwIP	Lightweight IP Stack	Recommended TCP/IP stack for STM32H7 implementations.
cJSON	Ultralight JSON Parser	Recommended JSON library for embedded VITA-SLC daemon side.

Appendix A Packet Size Reference Table

IP datagram sizes for all supported encoding depths and pair counts. Values assume standard Ethernet 1500-byte MTU. IP header = 20 bytes, UDP header = 8 bytes, VITA header = 28 bytes.

Encoding	Bytes/pair	360 pairs (default)	256 pairs	180 pairs	MTU safe at 360?
u8 (8-bit)	2	776 B	556 B	416 B	Yes ✓
int16 (16-bit)	4	1496 B	1076 B	776 B	Yes ✓ (4 B margin)
int24 (24-bit)	8	2936 B	2076 B	1496 B	No X — must reduce pairs
float32 (32-bit)	8	2936 B	2076 B	1496 B	No X — must reduce pairs

NOTE

For int24 and float32 encoding, the device MUST reduce samples_per_packet so that the IP datagram stays at or below 1500 bytes. The maximum for int24 and float32 is 180 pairs (1496 bytes). The actual samples_per_packet is returned in the SLC-CP open_stream response; clients SHALL use this value for all calculations.

Appendix B Profile Comparison

Feature	VITA-A/1.0 (IF Data)	VITA-A/1.0-CTX (Context)
Packet Type	0x1 (IF Data + Stream ID)	0x4 (IF Context + Stream ID)
Class Identifier	Mandatory. OUI=0x402814, PCC=0x0001.	Absent. CID bit=0.
TSI / TSF	01b / 10b (UTC + picoseconds)	00b / 00b (No Timestamp)
CIF0	Not present.	0x00000082 (always)
Reference Level	Not present.	16.16 signed fixed-point dBm at offset +12.
Payload Format	Not present.	Two words at offset +16/+20. Constant for a given mode.
Packet size	7 + samples_per_packet words	6 words (24 bytes, fixed)
Emitted when	Continuously during active stream.	Stream start; gain change; mode change. Max 1/100ms during AGC.
Packet Count	0..15, separate per stream.	0..15, separate from IF Data counter.
Byte order	Big-endian throughout.	Big-endian throughout.
Tier required	Tier 1 (all profiles).	Tier 2 (Standard and above).

Appendix C Implementation Checklist

Tier 1: minimum IF Data packet sender — checklist

Item
☐ VRT Header constant 0x18600000 computed for VITA-A/1.0 (type=1, CID=1, TSI=01b, TSF=10b, no trailer).
☐ Packet Count maintained as modulo-16 counter per stream. Incremented after each IF Data packet.
☐ Packet Size field set to 7 + payload_words (where payload_words = samples_per_packet for int16; samples_per_packet/2 for u8).
☐ Stream Identifier set from vita.stream_id in the SLC-CP open_stream response.
☐ Class ID Word 0 = 0x00402814 (constant).
☐ Class ID Word 1 = 0x00000001 (constant).
☐ Timestamp Integer updated once per second from RTC or free-running counter.
☐ sample_count reset to 0 on each start_rx call.
☐ Timestamp Fractional computed using integer arithmetic: $(\text{sample_count} \% \text{fs_hz}) * 10^{12} / \text{fs_hz}$.
☐ I/Q payload written in big-endian byte order.
☐ Payload interleaved: I0 Q0 I1 Q1 ... (not I0 I1 I2 ... Q0 Q1 Q2 ...).
☐ IP datagram size verified to be ≤ 1500 bytes.
☐ UDP sent to destination IP and port from SLC-CP open_stream request.
☐ Stream stops immediately on TCP control connection loss.

Tier 2: adds Context packet sender — additional checklist

Item
☐ Context packet counter (separate from IF Data counter) initialised to 0 at stream open.
☐ Context packet sent before first IF Data packet after start_rx.
☐ Context packet sent before first IF Data packet after any direct_sampling mode change.
☐ Context packet sent before first IF Data packet after any gain control change.
☐ VRT Header for Context: 0x40000006 (count <= 16). Type=0x4, no CID, no timestamp, size=6.
☐ CIF0 = 0x00000082 (constant).
☐ Reference Level encoded as 16.16 signed fixed-point. Integer part = floor(dBm), fraction = (dBm - floor) * 65536.
☐ Reference Level placed at offset +12 (before Payload Format at +16).
☐ Payload Format Word 0 = 0x0F8700E0 for complex IQ; 0x0F0700E0 for real.

Item
☐ Payload Format Word 1 = 0x00000000 (constant).
☐ current_reference_level_dbm updated on each gain change: new = old - gain_delta_dB.
☐ AGC gain-change handler emits Context packet (rate-limited to 1 per 100 ms).
☐ Reference Level initialised from factory calibration table or nominal value at stream open.